

Claude Extended Thinking

A Beginner's Guide to Fundamentals

With Python code examples · Source: Anthropic official docs

What is it?

How it works

budget_tokens

Streaming

Best Practices

1. What Is Extended Thinking?

Normally when you send a message to Claude, it reads your question and immediately starts writing its reply. This works great for simple tasks. But for hard problems — like solving a tricky math puzzle, designing a system, or analysing a complex document — jumping straight to an answer can miss important steps.

Extended thinking gives Claude a private scratchpad where it can *think through the problem step by step* before writing its final answer. Think of it like the difference between a student who blurts out a guess versus one who works through the problem on rough paper first — the second approach produces a much more reliable answer.

The analogy

Without Extended Thinking	With Extended Thinking
You ask a question → Claude answers immediately	You ask a question → Claude thinks privately → Claude writes a careful answer
Like answering off the top of your head	Like solving on a whiteboard first, then explaining
Fast, good for simple tasks	Slower, but much better for hard tasks

When should you use it?

Good use case	Example
Complex mathematics	Multi-step proofs, calculus, number theory
Difficult coding problems	Algorithm design, debugging intricate logic
Deep analysis	Analysing long documents, research synthesis
Strategic reasoning	Decision making with many trade-offs
Hard logical puzzles	Problems that need careful step-by-step deduction

- **Simple tasks don't need it.** For a quick question like "What is the capital of France?", extended thinking just adds latency and cost. Use it selectively for genuinely hard problems.

2. How Extended Thinking Works

When extended thinking is enabled, Claude produces two types of content blocks in its response:

- ① **A thinking block** — Claude's internal reasoning (like rough working)
- ② **A text block** — Claude's final answer to your question

What the response looks like (simplified)

RESPONSE STRUCTURE

```
# When extended thinking is on, the response looks like this:
{
  "content": [
    {
      "type": "thinking", # <-- Block 1: Claude's private reasoning
      "thinking": "Let me work through this step by step...
      First, I need to consider...
      So the answer must be...",
      "signature": "WaUjzkypQ2mU..." # encrypted verification token
    },
    {
      "type": "text", # <-- Block 2: The final answer
      "text": "Based on my analysis, the answer is 42."
    }
  ]
}
```

response.content is a list — thinking block first, text block second

What is 'summarized thinking'?

Claude 4 models don't show you Claude's *raw* thinking — they show a **summarised version** of it. This still gives you full intelligence benefit (Claude reasoned deeply), but the thinking text you receive is a condensed version of what actually happened internally. You are billed for the full thinking tokens, not just the summary.

- **Important:** The billed token count will NOT match what you see. Claude might think for 5,000 tokens internally, but the summarised thinking block you receive might only be 200 tokens. You still pay for the 5,000.

What is the 'signature' field?

The **signature** is an encrypted token that Anthropic uses to verify that the thinking block was genuinely produced by Claude. You don't need to read or interpret it — just pass it back unchanged if you're having a multi-turn conversation. Never modify it.

3. Enabling Extended Thinking — Basic Usage

To turn on extended thinking, add a **thinking** parameter to your `client.messages.create()` call. That's the only change needed.

Step 1 — Install the library

TERMINAL

```
pip install anthropic
```

Step 2 — Your first extended thinking call

EXAMPLE 1 — Basic extended thinking

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

response = client.messages.create(
    model="claude-sonnet-4-6", # model that supports extended thinking
    max_tokens=16000, # must be larger than budget_tokens
    thinking={
        "type": "enabled", # turn on extended thinking
        "budget_tokens": 10000 # max tokens Claude can use to think
    },
    messages=[
        {
            "role": "user",
            "content": "Are there an infinite number of prime numbers where n mod 4 == 3?"
        }
    ]
)

# Loop through content blocks in the response
for block in response.content:
    if block.type == "thinking":
        print("=== CLAUDE'S THINKING ===")
        print(block.thinking)
    elif block.type == "text":
        print("\n=== FINAL ANSWER ===")
        print(block.text)
```

Source: platform.claude.com/docs/en/build-with-claude/extended-thinking

Understanding the two key parameters

Parameter	What it does	Example value
thinking.type	Turns extended thinking on or off	"enabled" or "disabled"
thinking.budget_tokens	Max tokens Claude can use for internal reasoning. Larger = more thorough but slower.	10000
max_tokens	Total output limit (thinking + final answer). Must be GREATER than budget_tokens.	66000

■

Rule: budget_tokens must always be less than max_tokens. If budget_tokens=10000 and max_tokens=8000 the API will return an error. A safe pattern: set max_tokens = budget_tokens + 4000 (room for the final answer).

4. Understanding budget_tokens

budget_tokens controls how much *thinking space* Claude gets. It's like the size of the whiteboard — a bigger whiteboard lets Claude work through more complex problems, but takes more time and costs more.

Budget size guide

Budget Size	Tokens	Good For	Typical Latency
Minimum	1,024	Forced thinking enabled, simple tasks	Fast
Small	4,000	Moderately complex questions	Moderate
Medium	10,000	Most use cases — recommended starting point	Moderate
Large	32,000	Very complex reasoning, research-level problems	Slow
Max (Opus)	128,000	Most demanding tasks — use batch API above 32K	Very slow

Example: Comparing different budgets

EXAMPLE 2 — Comparing budget sizes

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

def ask_with_budget(question: str, budget: int) -> str:
    """Ask Claude a question with a specific thinking budget."""
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=budget + 4000, # always leave room for the answer
        thinking={
            "type": "enabled",
            "budget_tokens": budget # how much to think
        },
        messages=[{"role": "user", "content": question}]
    )
    # Extract just the text answer
    for block in response.content:
        if block.type == "text":
            return block.text
    return ""

question = "What is the most efficient sorting algorithm for nearly-sorted data and why?"
```

```
# Try different budget sizes
small_answer = ask_with_budget(question, budget=2000)
medium_answer = ask_with_budget(question, budget=10000)

print("Small budget answer:")
print(small_answer)
print("\nMedium budget answer:")
print(medium_answer)
```

■ **Tip:** Claude may not always use the full budget. If the problem is simpler than expected, Claude stops thinking early. The budget is a ceiling, not a requirement. Start with 10,000 for most tasks — you can tune up or down from there.

Checking how many thinking tokens were actually used

EXAMPLE 3 — Inspecting token usage

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": "Explain the P vs NP problem simply."}]
)

# Token usage breakdown
usage = response.usage
print(f"Input tokens: {usage.input_tokens}")
print(f"Output tokens: {usage.output_tokens} (this is what you're billed for)")

# Thinking-specific breakdown (how much of output was thinking)
if hasattr(usage, "output_tokens_details"):
    thinking_tokens = usage.output_tokens_details.thinking_tokens
    text_tokens = usage.output_tokens - thinking_tokens
    print(f" - Thinking tokens: {thinking_tokens}")
    print(f" - Text tokens: {text_tokens}")
```

5. Reading the Response Correctly

Because the response contains multiple block types, you must loop through **response.content** and handle each block by its **type**. Here are the patterns you'll use most often:

Pattern A — Get only the final text answer

EXAMPLE 4 — Extract only the final answer

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content": "What is 17 * 83?"}]
)

# Pattern A: skip thinking, only use the text answer
answer = ""
for block in response.content:
    if block.type == "text":
        answer = block.text
        break

print(f"Answer: {answer}")
# Output: Answer: 17 * 83 = 1,411
```

Pattern B — Show both thinking and the answer

EXAMPLE 5 — Show thinking + answer

```
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content": "Is 97 a prime number?"}]
)

# Pattern B: display thinking AND the answer
for block in response.content:
    if block.type == "thinking":
        print("--- Claude's Thinking Process ---")
```

```
print(block.thinking)

print("-" * 40)

elif block.type == "text":
    print("Final Answer:", block.text)
```

Pattern C — Collect all content into a dict

EXAMPLE 6 — Helper function to parse the response

```
def parse_response(response) -> dict:
    """Return a dict with keys 'thinking' and 'answer'."""
    result = {"thinking": None, "answer": None}

    for block in response.content:
        if block.type == "thinking":
            result["thinking"] = block.thinking
        elif block.type == "text":
            result["answer"] = block.text

    return result

# Usage
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 8000},
    messages=[{"role": "user", "content": "Explain why 0.1 + 0.2 != 0.3 in Python"}]
)

parsed = parse_response(response)
print("Thinking:", parsed["thinking"][:200], "...") # first 200 chars
print("Answer:", parsed["answer"])
```

6. Streaming Extended Thinking

With streaming, you receive the thinking and answer *as they are generated*, word by word — instead of waiting for the entire response. This is useful when the thinking budget is large (e.g. 16k+ tokens), because otherwise you'd wait a long time with no output.

■ **Required for large budgets:** If `max_tokens > 21,333`, the Anthropic SDK *requires* you to use streaming to avoid HTTP timeout errors on long requests.

Streaming with extended thinking

EXAMPLE 7 — Streaming extended thinking

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# Use client.messages.stream() instead of client.messages.create()
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{
        "role": "user",
        "content": "What is the greatest common divisor of 1071 and 462?"
    }]
) as stream:

    current_block_type = None

    for event in stream:

        # A new content block is starting (thinking or text)
        if event.type == "content_block_start":
            current_block_type = event.content_block.type
            if current_block_type == "thinking":
                print("\n[Thinking...]", end="", flush=True)
            elif current_block_type == "text":
                print("\n[Answer]", end="", flush=True)

        # Receiving a chunk of content
        elif event.type == "content_block_delta":
            if event.delta.type == "thinking_delta":
                print(event.delta.thinking, end="", flush=True)
            elif event.delta.type == "text_delta":
```

```
print(event.delta.text, end="", flush=True)

# Block finished

elif event.type == "content_block_stop":
    print("\n")
```

Source: platform.claude.com/docs/en/build-with-claude/extended-thinking

Simpler streaming — just get the final message

If you don't need to process events one by one, use `.get_final_message()` inside the stream context. You still use streaming under the hood (avoiding timeouts), but you get back the complete response at the end.

EXAMPLE 8 — Streaming without handling individual events

```
# Simpler streaming: get the full message at the end
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=30000,
    thinking={"type": "enabled", "budget_tokens": 25000},
    messages=[{"role": "user", "content": "Design a simple rate limiter in Python"}]
) as stream:
    response = stream.get_final_message() # waits for full response

# Now use response like a normal response object
for block in response.content:
    if block.type == "text":
        print(block.text)
```

7. Supported Models

Extended thinking is not available on every Claude model. Here's a quick reference:

Model	Extended Thinking Support	Recommended Approach
claude-sonnet-4-6	✓ Supported	Use thinking: {type: 'enabled', budget_tokens: N}
claude-opus-4-6	✓ Supported (deprecated manual thinking)	Prefer adaptive thinking
claude-opus-4-7	✗ Manual not supported	Use adaptive: {type: 'adaptive'}
claude-opus-4-8	✗ Manual not supported	Use adaptive: {type: 'adaptive'}
claude-haiku-4-5	✓ Supported	Use thinking: {type: 'enabled', budget_tokens: N}

What is adaptive thinking? (for Opus 4.7+)

Newer Opus models use **adaptive thinking** instead of manual extended thinking. With adaptive thinking, Claude decides *by itself* when and how much to think — you don't set a `budget_tokens` value. The API parameter looks like this:

EXAMPLE 9 — Adaptive thinking (Opus 4.7+)

```
# For Claude Opus 4.7 and Opus 4.8 – use adaptive thinking
response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={
        "type": "adaptive" # Claude decides when/how much to think
        # No budget_tokens needed!
    },
    messages=[{"role": "user", "content": "Solve this: 3x^2 - 5x + 2 = 0"}]
)
```

- For beginners: stick with **claude-sonnet-4-6** and manual extended thinking (type: 'enabled', budget_tokens: N). It's the most straightforward to learn with and clearly shows you the thinking/answer structure.

8. Key Limitations to Know

Extended thinking has some restrictions. Trying to use incompatible features together will cause an API error. Here's what you need to know:

Feature	Compatible with Extended Thinking?	Notes
temperature parameter	✗ Not supported	Removed automatically; use defaults
top_k parameter	✗ Not supported	Removed automatically
top_p parameter	■ Limited	Only values between 0.95 and 1
Response pre-filling	✗ Not supported	Cannot pre-fill Claude's response
Forced tool use	✗ Not supported	Use auto or none tool_choice only
Tool use (auto)	✓ Supported	Full tool use works fine
Streaming	✓ Supported	Required for max_tokens > 21,333
Prompt caching	✓ Supported	See caching notes in docs
max_tokens = 0	✗ Not supported	Cannot be used for cache pre-warming

Safe, complete minimal example (no incompatible params)

EXAMPLE 10 — Safe minimal template

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    # temperature= NOT included (not compatible)
    # top_k= NOT included (not compatible)
    thinking={
        "type": "enabled",
        "budget_tokens": 10000
    },
    messages=[{
        "role": "user",
        "content": "Write a Python function to check if a number is a perfect number."
    }]
)
```

```
)  
  
for block in response.content:  
    if block.type == "text":  
        print(block.text)
```

9. Best Practices

1. Start with `budget_tokens=10000`

This is the official recommended starting point. Adjust up or down based on results. Don't jump straight to 32k — it increases latency and cost without always improving quality.

2. Always set `max_tokens > budget_tokens`

A safe formula: `max_tokens = budget_tokens + 4000`. This leaves room for Claude's final answer after thinking.

3. Use streaming for large budgets

If `budget_tokens > 16,000`, use `client.messages.stream()` to avoid HTTP timeouts and to show progress to users while Claude thinks.

4. Don't use it for simple tasks

Extended thinking adds latency and cost. For quick lookups, translation, or simple Q&A; — just use normal messages without thinking.

5. Check `thinking_tokens` in usage

Monitor `response.usage.output_tokens_details.thinking_tokens` to see how much thinking actually happened. If Claude barely uses the budget, the task is simple enough to not need extended thinking.

6. Never modify thinking blocks

In multi-turn conversations, pass thinking blocks back to the API exactly as you received them. Modifying or discarding them breaks the reasoning continuity.

7. Use batch API for very large budgets

For `budget_tokens` above 32k, use Anthropic's batch processing API to avoid connection timeouts on very long requests.

- **General rule of thumb:** If your task takes a human expert more than a few minutes to think through carefully, extended thinking will likely help Claude produce a better answer. If a human could answer it in seconds, save the cost and latency.

10. Complete Working Example

Here is a complete, self-contained Python script that uses extended thinking to solve a difficult problem. Copy and run it directly (replace the API key).

EXAMPLE 11 — Complete working script

```
"""
Complete extended thinking example.
Asks Claude a complex question and shows both thinking and answer.
"""

import anthropic

def ask_claude_with_thinking(question: str, budget: int = 10000) -> dict:
    """
    Send a question to Claude with extended thinking enabled.

    Args:
    question: The question to ask
    budget: Max tokens for Claude's thinking (default 10000)

    Returns:
    dict with 'thinking' and 'answer' keys
    """
    client = anthropic.Anthropic(api_key="YOUR_API_KEY")

    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=budget + 4000, # leave room for the answer
        thinking={
            "type": "enabled",
            "budget_tokens": budget
        },
        messages=[{"role": "user", "content": question}]
    )

    result = {"thinking": None, "answer": None, "thinking_tokens": 0}

    for block in response.content:
        if block.type == "thinking":
            result["thinking"] = block.thinking
        elif block.type == "text":
            result["answer"] = block.text

    # Token usage
    if hasattr(response.usage, "output_tokens_details"):
```

```
result["thinking_tokens"] = (
    response.usage.output_tokens_details.thinking_tokens
)

return result

if __name__ == "__main__":
    question = """
    I have a list of 1 million numbers. I want to find all pairs of numbers
    that sum to a target value T. What is the most efficient algorithm,
    and what is its time and space complexity?
    """

    print(f"Question: {question.strip()}\n")
    print("Asking Claude with extended thinking (budget: 8,000 tokens)...")
    print("=" * 60)

    result = ask_claude_with_thinking(question, budget=8000)

    print(f"Thinking tokens used: {result['thinking_tokens']}")
    print("\n--- Claude's Thinking (summarised) ---")
    if result["thinking"]:
        # Show first 500 chars of thinking
        preview = result["thinking"][:500]
        print(preview + "..." if len(result["thinking"]) > 500 else preview)

    print("\n--- Final Answer ---")
    print(result["answer"])
```

Copy, replace YOUR_API_KEY, and run directly

11. Extended Thinking vs. Chain of Thought

Both approaches make an AI reason step-by-step before answering. But they differ in **where the reasoning happens**, **who controls it**, and **how reliable it is**. Understanding the difference helps you pick the right tool.

Chain of Thought (CoT)?

A prompting technique — you ask the model to think out loud

Chain of Thought is not a feature — it's a **prompting strategy**. You simply instruct the model in plain text to reason step by step, and it writes its reasoning directly into the response as visible text, mixed in with the answer.

EXAMPLE 12 — Chain of Thought via prompting

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# Chain of Thought: the reasoning instruction goes in the prompt
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=2048,
    # NO thinking parameter – this is plain prompting
    messages=[{
        "role": "user",
        "content": """Solve this step by step:
A train travels 120 km at 60 km/h, then 80 km at 40 km/h.
What is the average speed for the whole journey?

Think through this carefully, showing each step."""
    }]
)

# The reasoning IS the text – no separate blocks
print(response.content[0].text)

# Output (example):
# Step 1: Time for first leg = 120 km / 60 km/h = 2 hours
# Step 2: Time for second leg = 80 km / 40 km/h = 2 hours
# Step 3: Total distance = 120 + 80 = 200 km
# Step 4: Total time = 2 + 2 = 4 hours
# Step 5: Average speed = 200 / 4 = 50 km/h
# Answer: The average speed is 50 km/h
```

Reasoning is written into the text response — no special API feature needed

Thinking?

A native model capability — reasoning happens in a protected internal process

Extended Thinking is a built-in Claude feature enabled through the API. Claude reasons in a separate, protected process *before* writing the response. The reasoning lives in its own **thinking block**, signed and verified by Anthropic. The final text block contains only the clean answer.

EXAMPLE 13 — Same problem with Extended Thinking

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# Extended Thinking: enabled via API parameter, not the prompt
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=14000,
    thinking={
        "type": "enabled", # <-- API feature, not a prompt trick
        "budget_tokens": 10000
    },
    messages=[{
        "role": "user",
        # Notice: no "think step by step" instruction needed in the prompt
        "content": """"A train travels 120 km at 60 km/h, then 80 km at 40 km/h.
        What is the average speed for the whole journey?"""
    }]
)

# Reasoning and answer are in SEPARATE blocks
for block in response.content:
    if block.type == "thinking":
        print("[Thinking block – Claude's internal reasoning]")
        print(block.thinking[:300], "...") # summarised reasoning
    elif block.type == "text":
        print("[Answer block – clean final response]")
        print(block.text)

# Output:
# [Thinking block] Let me work through this carefully...
# Time1 = 120/60 = 2h, Time2 = 80/40 = 2h, Total = 4h...
```

```
# [Answer block] The average speed is 50 km/h.
```

Reasoning lives in a separate signed block — the answer block is clean

Side-by-side comparison

	Chain of Thought (CoT)	Extended Thinking
How enabled	Write 'think step by step' in your prompt	API param: thinking={type:'enabled'}
Where reasoning lives	Inside the response text — mixed with answer	Separate thinking block
Works on any model	Yes — any LLM that follows instructions	No — Claude-specific, select models
Tamper-proof?	No — just text, can be influenced by prompts	Yes — cryptographically signed
Visible to you	Fully visible as plain text	Summarised (Claude 4 models)
Reasoning quality	Good — depends on prompt quality	Better — deeper, uninterrupted
Token cost	You pay for reasoning tokens as output	Same, billed on full internal tokens
Control mechanism	You shape it via prompt wording	You set a budget; model decides how
Can be 'tricked'?	Yes — adversarial prompts can disrupt it	Harder — internal process is protected
Response structure	One text block containing everything	ThinkingBlock + TextBlock
Best for	Moderate tasks, any model, quick setup	Hard problems requiring deep reasoning

Can you combine both?

Yes — and it often works well. You can enable Extended Thinking *and* still write a structured prompt that guides how Claude approaches the problem. Extended Thinking handles the depth; your prompt handles the direction.

EXAMPLE 14 — Combining structured prompting with Extended Thinking

```
# Combining CoT-style prompting WITH Extended Thinking
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{
        "role": "user",
        "content": ""You are a senior software engineer.
Approach this problem systematically:
1. First identify edge cases
2. Then consider time and space complexity
```

```
3. Finally write the solution

Problem: Given a binary tree, find the longest path between any two nodes."""
}]
)

for block in response.content:
    if block.type == "text":
        print(block.text)
```

Which should you use?

Situation	Recommended approach
Simple or moderate question	Neither — just ask normally
Moderate complexity, any LLM	Chain of Thought (prompt-based)
Hard problem, using Claude	Extended Thinking
Need to see full reasoning as text	Chain of Thought
Need most reliable / tamper-proof answer	Extended Thinking
Using GPT or Gemini	Chain of Thought (ET is Claude-only)
Both prompt control + deep reasoning	Combine both

- **Key insight:** Chain of Thought is reasoning *written out loud as text*. Extended Thinking is reasoning happening in a *protected internal layer* that the model was specifically trained to use. The second goes deeper and is harder to disrupt — but costs more and only works on Claude.

Summary

Here is everything covered in this guide at a glance:

Concept	Key Point
What it is	A scratchpad for Claude to reason step-by-step before answering
When to use it	Complex maths, coding, analysis — NOT for simple questions
How to enable	Add <code>thinking={'type':'enabled','budget_tokens':N}</code> to your call
budget_tokens	Max tokens Claude can use to think. Start at 10,000.
max_tokens rule	Always set <code>max_tokens > budget_tokens</code> (add ~4,000 extra)
Response structure	<code>response.content = [ThinkingBlock, TextBlock]</code>
Get the answer	Loop through content, find <code>block.type == 'text'</code>
Streaming	Use <code>client.messages.stream()</code> — required for <code>max_tokens > 21,333</code>
Billing	You pay for full thinking tokens even though you see a summary
Key restriction	No temperature, no top_k, no response pre-fill
Best model to learn	claude-sonnet-4-6 with <code>type='enabled'</code>
vs Chain of Thought	CoT = prompt trick, reasoning in text. ET = native feature, protected internal layer

- You now understand the fundamentals of Claude's extended thinking. Start with **Example 1** (basic call), then try **Example 11** (complete script). Tune `budget_tokens` based on your task complexity and latency requirements.

Source: platform.claude.com/docs/en/build-with-claude/extended-thinking